



MODÉLISATION ET PROTOCOLE EXPÉRIMENTAL

**Apprentissage par renforcement d'une politique de navigation
mono-agent avec l'algorithme SAC**

SUJET DU STAGE : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

AUTEURE : Sayan Suos

Table des matières

Introduction	2
1 Modélisation du problème	3
1.1 Espace des états	3
1.2 Espace des actions	4
1.3 Fonction de récompense	4
2 Apprentissage avec l’algorithme Soft Actor-Critic	6
2.1 Principe général de SAC	6
2.2 Apprentissage de réseaux	7
2.3 Architecture des réseaux	9
3 Protocole expérimental	12
3.1 Curriculum d’entraînement	12
3.2 Environnements de test	17
3.3 Critères d’évaluation	19
4 Résultats	21
4.1 Validation de l’apprentissage	21
4.2 Analyse des scénarios d’entraînement	21
4.3 Évaluation de la politique	22
Glossaire	23
A Annexes	27
A.1 Pseudo-code de l’algorithme SAC	27
A.2 Compléments sur l’entraînement	27

Introduction

Le transport de bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire soulève des problématiques de navigation multi-agents en environnement dynamique. Au sein de notre projet, l'objectif est de minimiser le temps de transport tout en garantissant l'absence de collision, dans un cadre décentralisé où les agents ne disposent que d'observations locales et ne communiquent pas explicitement entre eux. Dans cette optique, un simulateur est actuellement développé afin de modéliser l'environnement étudié et d'expérimenter différentes approches d'apprentissage par renforcement.

Dans un premier temps, nous nous intéressons à un cadre plus général de navigation autonome, en considérant des robots mobiles autonomes (AMR). Cette approche permet d'étudier et de valider les méthodes d'apprentissages dans un environnement plus libre, avant de les adapter progressivement aux contraintes spécifiques du contexte aéroportuaire.

Dans cette première phase expérimentale, nous choisissons d'étudier un algorithme classique d'apprentissage par renforcement profond afin de prendre en main le simulateur et d'implémenter les principaux éléments du problème. Plus précisément, nous entraînons un agent unique dans un environnement comportant des obstacles statiques et dynamiques à l'aide de l'algorithme Soft Actor-Critic (SAC). Ce choix est motivé par le fait que SAC est une méthode off-policy, ce qui permet une meilleure efficacité d'apprentissage. De plus, elle est particulièrement adaptée aux espaces d'actions continues et est largement utilisée dans la littérature pour ses bonnes performances et sa stabilité ([Haarnoja et al., 2018](#); [OpenAI, 2023](#)).

1 Modélisation du problème

Cette section présente la modélisation du problème de navigation sous la forme d'un processus de décision de Markov (MDP). Les différentes composantes du modèle, à savoir l'espace des états, l'espace des actions et la fonction de récompense, sont détaillées dans les sous-sections suivantes.

Le MDP est représenté par le tuple (S, A, P, R, γ) où :

- S est l'ensemble des états qui permettent à l'agent de sélectionner une action,
- A est l'ensemble des actions qui correspondent aux décisions de l'agent,
- $P : S \times A \times S \rightarrow [0, 1]$ est la probabilité de transition entre les états, qui modélise la densité de probabilité du prochain état s_{t+1} , conditionnellement à l'état courant s_t et l'action a_t ,
- $R : S \times A \rightarrow [r_{\min}, r_{\max}]$ est la fonction de récompense qui attribue une valeur à chaque transition,
- et γ est le facteur d'actualisation qui modélise l'importance du futur.

L'objectif de l'agent est de rejoindre une position cible, tout en évitant les collisions. Dans cette étude, le temps est discrétisé et l'on considère qu'un instant t correspond à une étape de simulation (step), défini comme une seconde. À chaque étape, l'agent observe son état, sélectionne une action et l'environnement évolue vers un nouvel état.

1.1 Espace des états

À chaque instant t , l'agent observe un état $s_t \in S$ composé de plusieurs informations qui décrivent son environnement local et sa dynamique :

$$s_t = (\mathbf{M}_t, d_t^g, \mathbf{e}_t, \mathbf{v}_t, \mathbf{o}_t).$$

- $\mathbf{M}_t \in \{0, 1\}^{K \times W \times H}$ représente l'environnement local de l'agent construit à partir des K dernières observations. Chaque observation est une carte encodée de taille $W \times H$, centrée sur l'agent. Une cellule vaut 1 lorsqu'elle est occupée par un obstacle et 0 sinon.
- $d_t^g \in \mathbb{R}^+$ représente la distance euclidienne entre la position courante de l'agent et la position cible :

$$d_t^g = \|\mathbf{p}_g - \mathbf{p}_t\|_2,$$

où $\mathbf{p}_t = (x_t, y_t)$ désigne la position courante de l'agent et $\mathbf{p}_g = (x_g, y_g)$ la position cible.

- $\Delta\theta_t^g$ désigne l'erreur angulaire entre l'orientation du robot et la direction de la cible :

$$\Delta\theta_t^g = \theta_t^g - \theta_t,$$

où θ_t^g désigne la direction de la cible depuis la position courante de l'agent, exprimée dans le repère global, et θ_t l'orientation courante de l'agent. Cette erreur est fournie au réseau sous la forme : $\mathbf{e}_t = (\cos(\Delta\theta_t^g), \sin(\Delta\theta_t^g))$.

- $\mathbf{v}_t = (v_t, \omega_t)$ représente la dynamique du robot, avec v_t sa vitesse linéaire et ω_t sa vitesse angulaire.
- $\mathbf{o}_t = (\cos(\theta_t), \sin(\theta_t))$ représente l'orientation du robot.

Les angles sont représentés à l'aide de leurs composantes trigonométriques afin d'éviter les discontinuités associées à une représentation angulaire directe. L'orientation θ_t est définie sur l'intervalle $[-\pi, \pi]$. Les commandes de contrôle sont contraintes par les limites physiques du robot, avec $v_t \in [v_{\min}, v_{\max}]$ et $\omega_t \in [\omega_{\min}, \omega_{\max}]$.

L'ensemble des paramètres liés au robot est récapitulé dans la Table 5.

1.2 Espace des actions

À chaque instant t , l'agent choisit une action continue $\mathbf{a}_t \in A$ qui correspond aux commandes de contrôle appliquées au robot :

$$\mathbf{a}_t = (v_t, \omega_t),$$

où v_t et ω_t désignent respectivement la vitesse linéaire et la vitesse angulaire du robot, avec $v_t \in [v_{\min}, v_{\max}]$ et $\omega_t \in [\omega_{\min}, \omega_{\max}]$.

1.3 Fonction de récompense

La fonction de récompense est définie comme la somme de plusieurs termes :

$$R = R_g + R_r + R_s + R_c,$$

où R_g regroupe les termes associés à la progression vers l'objectif, R_r pénalise les rotations importantes, R_s pénalise la proximité des obstacles et R_c pénalise les collisions.

1.3.1 Récompense de progression

À chaque instant t , la progression de l'agent vers sa cible est définie par la variation de la distance à l'objectif :

$$\Delta d_t^g = d_{t-1}^g - d_t^g.$$

Une valeur positive indique alors que l'agent s'est rapproché de sa cible entre $t - 1$ et t .

La récompense de progression est initialement calculée à partir de cette progression et de la vitesse linéaire du robot. Afin de favoriser la sécurité, lorsque la distance d_t^o entre le robot et l'obstacle le plus proche devient inférieure au seuil de sécurité ξ_s , l'ensemble de cette récompense est réduit d'un facteur $r_{\text{reduction}}$, avec $0 < r_{\text{reduction}} \leq 1$. On obtient ainsi :

$$R_g = \begin{cases} \beta_1 \times \Delta d_t^g + r_{\text{linear}} \times v_t + r_{\text{step}} & \text{si } d_t^o \geq \xi_s, \\ r_{\text{reduction}} (\beta_1 \times \Delta d_t^g + r_{\text{linear}} \times v_t) + r_{\text{step}} & \text{si } d_t^o < \xi_s, \end{cases}$$

où $r_{\text{linear}} > 0$ est le coefficient associé au bonus de vitesse linéaire, $r_{\text{step}} < 0$ est une pénalité constante appliquée à chaque étape et $r_{\text{reduction}}$ est le facteur de réduction utilisé à proximité d'un obstacle.

Cette formulation encourage l'agent à progresser rapidement vers sa cible tout en diminuant l'intérêt d'une vitesse élevée lorsqu'il évolue à proximité d'un obstacle. La pénalité constante appliquée à chaque étape incite également l'agent à accomplir sa mission en un nombre limité d'actions.

Lorsque l'objectif est atteint, un bonus supplémentaire est ajouté :

$$R_g \leftarrow R_g + r_{\text{goal}}, \quad \text{où } r_{\text{goal}} > 0.$$

L'objectif est considéré comme atteint lorsque $d_t^g \leq \xi_g$, où ξ_g désigne le seuil d'atteinte de l'objectif.

1.3.2 Pénalité de rotation

Afin d'éviter les rotations excessives, une pénalité quadratique est appliquée à la vitesse angulaire :

$$R_r = \beta_2 \times r_{\text{angular}} \times \omega_t^2, \quad \text{où } r_{\text{angular}} < 0.$$

L'utilisation du carré de la vitesse angulaire augmente la pénalité pour les rotations importantes et favorise ainsi des trajectoires plus fluides.

1.3.3 Pénalité de sécurité

Lorsque le robot entre dans la zone de sécurité définie par $d_t^o < \xi_s$, une pénalité supplémentaire est appliquée :

$$R_s = \begin{cases} \beta_3 \times r_{\text{safety}} \times \left(\frac{1}{\tilde{d}_t^2} - \frac{1}{\xi_s^2} \right) & \text{si } d_t^o < \xi_s, \\ 0 & \text{sinon,} \end{cases}$$

avec $\tilde{d}_t = \max(0.1, d_t^o)$ et $r_{\text{safety}} < 0$.

Cette pénalité est nulle à la limite du seuil de sécurité et augmente rapidement lorsque le robot se rapproche d'un obstacle. La distance est bornée inférieurement à 0.1 afin d'éviter une divergence du terme $1/\tilde{d}_t^2$ lorsque la distance devient très faible.

1.3.4 Pénalité de collision

Enfin, lorsqu'une collision est détectée, une forte pénalité est attribuée :

$$R_c = r_{\text{collision}}, \quad \text{où } r_{\text{collision}} < 0.$$

L'état de collision est déterminé par le simulateur lorsque la distance à l'obstacle le plus proche devient inférieure ou égale au seuil de collision ξ_c , soit $d_t^o \leq \xi_c$.

Les coefficients de cette fonction de récompense ont été déterminés empiriquement au cours des différentes phases d'expérimentation, en s'appuyant sur les principes proposés dans la littérature (Long et al., 2018; Choi et al., 2021; Huang et al., 2024). Les valeurs retenues sont récapitulées dans la Table 5. **L'objectif est d'obtenir un compromis entre progression vers la cible, rapidité de déplacement, fluidité des trajectoires et sécurité vis-à-vis des obstacles.**

2 Apprentissage avec l’algorithme Soft Actor-Critic

Cette section présente l’algorithme Soft Actor-Critic (SAC) retenu pour l’apprentissage de la politique de navigation, ainsi que les différentes composantes de son architecture et de son processus d’apprentissage.

Les principaux paramètres utilisés pour l’entraînement du modèle ainsi que les hyperparamètres de l’algorithme SAC sont récapitulés dans la Table 5. Le pseudo-code de l’algorithme est résumé dans l’Algorithme 1.

2.1 Principe général de SAC

Avant de détailler l’apprentissage des réseaux, il est nécessaire de présenter les principales notions utilisées par SAC ainsi que l’objectif de maximum d’entropie sur lequel repose l’algorithme.

2.1.1 Politique, trajectoires et transitions

Une politique $\pi(\mathbf{a}_t|\mathbf{s}_t)$ définit le comportement de l’agent en associant à chaque état une distribution de probabilité sur les actions. En interagissant avec l’environnement, cette politique génère une trajectoire :

$$\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \dots).$$

On note $\rho_\pi(\mathbf{s})$ la distribution des états visités lorsque l’agent suit la politique π , et $\rho_\pi(\mathbf{s}, \mathbf{a})$ la distribution conjointe des couples état-action rencontrés au cours de ces trajectoires.

Dans le cas de SAC, les interactions avec l’environnement génèrent des transitions de la forme :

$$(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d),$$

où $\mathbf{s}'$ désigne l’état suivant obtenu après l’exécution de l’action $\mathbf{a}$ dans l’état $\mathbf{s}$, r la récompense associée à cette transition, et $d \in \{0, 1\}$ indique si l’état suivant est terminal. Ces transitions sont stockées dans un replay buffer $\mathcal{D}$, puis réutilisées pour entraîner les réseaux de neurones. SAC est ainsi un algorithme off-policy, puisqu’il peut apprendre à partir de transitions collectées précédemment.

Les méthodes actor-critic reposent sur deux composantes : l’actor paramètre la politique π_θ et sélectionne les actions, tandis que le critic estime la qualité des actions au moyen d’une fonction de valeur d’action $Q_\phi(\mathbf{s}, \mathbf{a})$.

Dans la suite, π désigne une politique de manière générale, tandis que π_θ désigne la politique paramétrée par l’actor network, de paramètres θ . De même, Q^π désigne la fonction de valeur d’action théorique associée à une politique π , tandis que Q_ϕ désigne son approximation par le critic network, de paramètres ϕ .

2.1.2 Objectif de maximum d'entropie

En apprentissage par renforcement, l'objectif classique consiste à maximiser la somme des récompenses futures. SAC généralise cet objectif en intégrant un terme d'entropie. Cela encourage la politique à conserver une certaine diversité dans ses actions :

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))) \right].$$

Ce principe correspond au principe de maximum entropy reinforcement learning utilisé par SAC.

L'entropie de la politique dans un état $\mathbf{s}$ est définie par :

$$\mathcal{H}(\pi(\cdot | \mathbf{s})) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot | \mathbf{s})} [-\log \pi(\mathbf{a} | \mathbf{s})].$$

Le paramètre $\alpha > 0$, appelé paramètre de température, contrôle le compromis entre maximisation de la récompense et exploration. Une valeur élevée de α accorde davantage d'importance à l'entropie, tandis qu'une valeur faible privilégie davantage les actions ayant une forte valeur estimée. Dans notre implémentation, α est fixé avant l'apprentissage.

Dans ce cadre, la fonction de valeur d'action $Q^\pi(\mathbf{s}, \mathbf{a})$ représente la valeur attendue d'une action $\mathbf{a}$ exécutée dans un état $\mathbf{s}$, en tenant compte des récompenses futures et de l'entropie de la politique. La fonction de valeur d'état $V^\pi(\mathbf{s})$ représente quant à elle la valeur attendue d'un état lorsque les actions suivantes sont sélectionnées selon la politique π .

Dans SAC, ces deux fonctions sont liées par :

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot | \mathbf{s})} [Q^\pi(\mathbf{s}, \mathbf{a}) - \alpha \log \pi(\mathbf{a} | \mathbf{s})]$$

La valeur d'un état dépend ainsi à la fois de la qualité des actions disponibles, représentée par Q^π , et de l'entropie de la politique.

2.2 Apprentissage de réseaux

L'apprentissage de SAC repose sur l'optimisation conjointe de l'actor network, qui paramètre la politique, et de deux critic networks, qui approximent les fonctions de valeur d'action. L'actor est optimisé à partir des estimations fournies par les critics, tandis que ces derniers sont entraînés à partir des transitions stockées dans le replay buffer.

2.2.1 Apprentissage de l'actor network

L'actor doit apprendre à sélectionner des actions qui maximisent la fonction de valeur tout en favorisant des politiques de forte entropie. L'objectif peut être écrit sous la forme :

$$\max_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_{\theta}} \left[\min_{i \in \{1, 2\}} Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - \alpha \log \pi_{\theta}(\mathbf{a} | \mathbf{s}) \right].$$

Le premier terme favorise les actions estimées comme avantageuses par les critics, tandis que le second favorise une politique suffisamment stochastique.

Dans SAC, la politique utilisée pour les actions continues est généralement une politique gaussienne. L'actor produit une moyenne $\mu_\theta(\mathbf{s})$ et un écart-type $\sigma_\theta(\mathbf{s})$. Une action est ensuite obtenue grâce au reparameterization trick (Kingma and Welling, 2022).

$$\mathbf{u} = \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s}) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Une fonction tangente hyperbolique est ensuite appliquée afin de borner l'action :

$$\tilde{\mathbf{a}}_\theta = \tanh(\mathbf{u}).$$

L'action obtenue appartient alors à l'intervalle $[-1, 1]$ et est ensuite remise à l'échelle afin de respecter les bornes de l'espace d'action. Cette transformation par $\tanh$, appelée squashed Gaussian policy, est celle utilisée dans SAC pour les espaces d'actions continus.

2.2.2 Apprentissage des critic networks

Le rôle du critic est d'approximer la fonction de valeur d'action $Q^\pi(\mathbf{s}, \mathbf{a})$, qui évalue la qualité d'une action $\mathbf{a}$ exécutée dans un état $\mathbf{s}$, en tenant compte des récompenses futures et de l'entropie de la politique.

Dans SAC, la valeur du prochain état tient également compte de l'entropie de la politique :

$$V^\pi(\mathbf{s}') = \mathbb{E}_{\mathbf{a}' \sim \pi(\cdot | \mathbf{s}')} [Q^\pi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi(\mathbf{a}' | \mathbf{s}')].$$

La fonction Q^π vérifie alors l'équation de Bellman :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = R(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim P(\cdot | \mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')]]$$

En combinant ces deux expressions, on obtient une version entropique de l'équation de Bellman, utilisée dans SAC :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim P(\cdot | \mathbf{s}, \mathbf{a}), \mathbf{a}' \sim \pi(\cdot | \mathbf{s}')} [R(\mathbf{s}, \mathbf{a}) + \gamma (Q^\pi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi(\mathbf{a}' | \mathbf{s}'))].$$

En pratique, cette espérance n'est pas calculée exactement. Une transition $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$ est échantillonnée dans le replay buffer $\mathcal{D}$, puis une action suivante $\tilde{\mathbf{a}}'$ est générée par la politique courante dans l'état $\mathbf{s}'$.

$$Q^\pi(\mathbf{s}, \mathbf{a}) \approx r + \gamma (Q^\pi(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}')), \quad \tilde{\mathbf{a}}' \sim \pi(\cdot | \mathbf{s}').$$

La cible d'apprentissage utilisée pour le critic est alors :

$$y = r + \gamma(1 - d) \left(\min_{i \in \{1, 2\}} Q_{\phi_i, \text{target}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi_\theta(\tilde{\mathbf{a}}' | \mathbf{s}') \right).$$

Le facteur $1 - d$ annule ainsi la contribution des récompenses futures lorsque l'état suivant est terminal.

Afin de limiter le biais de surestimation de la fonction de valeur, les versions modernes de SAC utilisent le principe du double-Q learning (Haarnoja et al., 2018; van Hasselt, 2010) : deux fonctions de valeur Q_{ϕ_1} et Q_{ϕ_2} sont apprises simultanément, et seule la plus petite estimation est utilisée.

Chaque critic est entraîné en minimisant l'erreur quadratique entre sa prédiction et cette cible :

$$L(\phi_i) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \sim \mathcal{D}} \left[(Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y)^2 \right].$$

Afin de stabiliser l'apprentissage, chaque critic network est associé à un réseau cible, dont les paramètres sont mis à jour progressivement selon la règle de soft update :

$$\phi_{i,\text{target}} \leftarrow (1 - \lambda)\phi_{i,\text{target}} + \lambda\phi_i,$$

où $\lambda \ll 1$ est le coefficient de mise à jour.

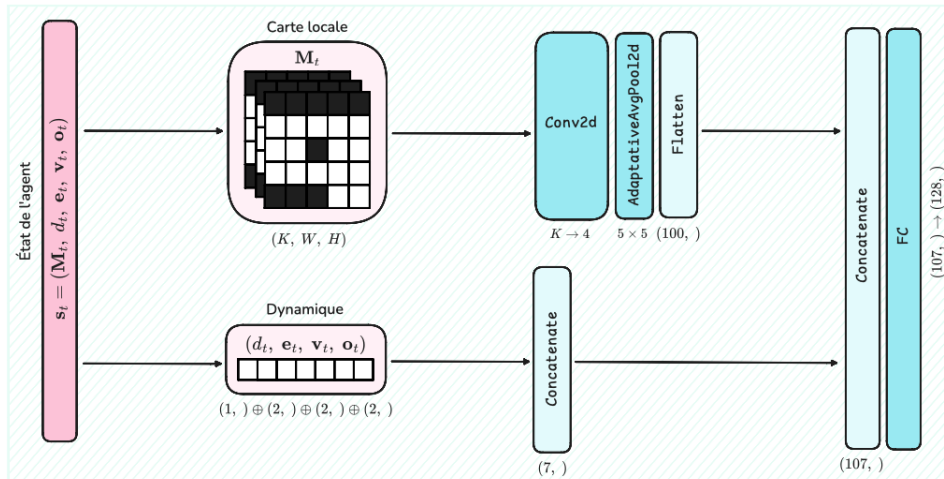
2.3 Architecture des réseaux

L'architecture utilisée repose sur un actor network et deux critic networks. Ces réseaux traitent à la fois la carte locale de l'agent et les autres composantes de son état. Deux réseaux cibles supplémentaires sont également associés aux critics afin de stabiliser l'apprentissage. Cette architecture s'inspire de Chen et al. (2020), qui utilise des cartes locales égocentriques pour l'apprentissage d'une politique de navigation.

2.3.1 Architecture de l'actor network

L'actor network paramètre la politique stochastique π_θ . Il prend en entrée l'observation de l'agent, composée de la carte locale, de la distance normalisée à la cible, de l'erreur d'orientation, des vitesses normalisées et de l'orientation du robot.

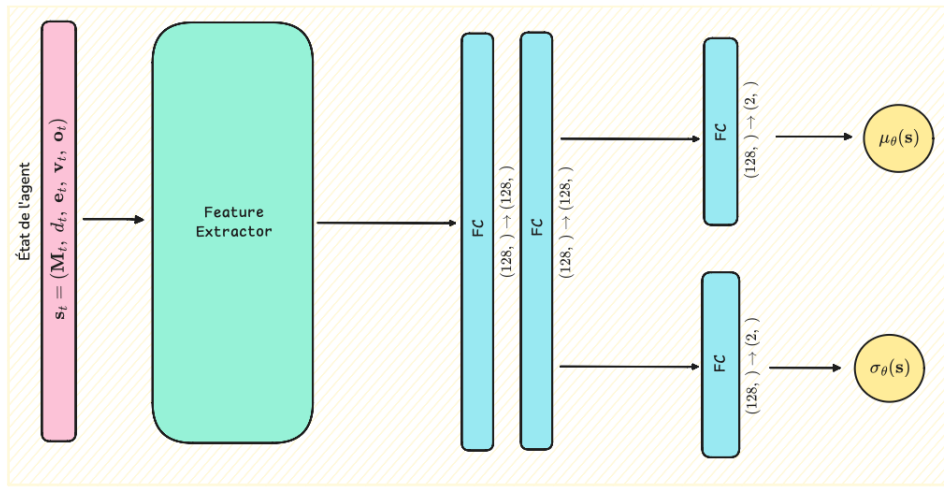
FIGURE 1 – Architecture générale de l'extracteur de caractéristiques convolutionnel



La carte locale est d'abord traitée par un extracteur de caractéristiques convolutionnel, illustré par la Figure 1. Celui-ci est composé d'une couche de convolution, suivie d'une fonction ReLU, puis d'un pooling adaptatif permettant d'obtenir une représentation de taille fixe. Cette représentation est ensuite aplatie et concaténée aux autres variables de l'état.

L'ensemble des informations est ensuite projeté dans un espace latent de dimension fixe. Ce vecteur latent est fourni à un réseau entièrement connecté composé de deux couches cachées, illustré par la Figure 2. La sortie du réseau est divisée en deux branches : une première branche prédit la moyenne $\mu_\theta(\mathbf{s})$ de la distribution d'action, tandis qu'une seconde branche prédit le logarithme de l'écart-type $\log \sigma_\theta(\mathbf{s})$. Le logarithme de l'écart-type est borné afin d'éviter des valeurs trop grandes ou trop faibles, ce qui stabilise l'apprentissage.

FIGURE 2 – Architecture générale de l'actor network



À partir de ces deux sorties, une action est échantillonnée selon une loi gaussienne à l'aide du reparameterization trick. Elle est ensuite transformée par une fonction tangente hyperbolique, puis remise à l'échelle afin de respecter les bornes de l'espace d'action. L'action finale correspond donc aux deux commandes du robot : la vitesse linéaire v_t et la vitesse angulaire ω_t .

En phase d'évaluation, l'action n'est plus échantillonnée aléatoirement : la moyenne de la distribution est utilisée directement avant l'application de la fonction tanh et de la remise à l'échelle. Cela rend la politique déterministe et permet d'obtenir un comportement plus stable.

2.3.2 Architecture des critic networks

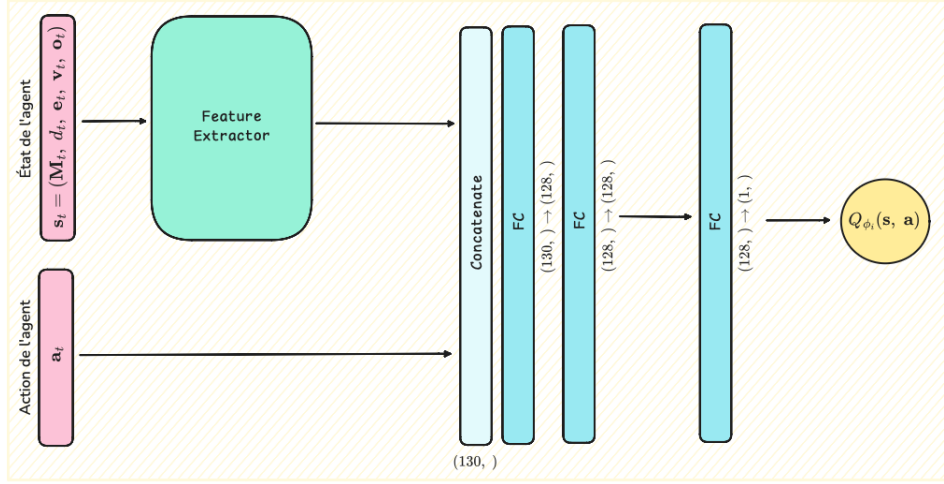
Les critic networks estiment la valeur d'une action donnée dans un état donné $Q_\phi(\mathbf{s}, \mathbf{a})$. Ces deux réseaux ont la même architecture, illustrée par la Figure 3, mais leurs paramètres sont appris séparément.

Comme pour l'actor, chaque critic commence par extraire une représentation latente de l'état. La carte locale est traitée par le même type d'extracteur convolutionnel, puis les caractéristiques obtenues sont concaténées avec les autres composantes de l'observation.

La différence principale avec l'actor est que le critic reçoit également l'action en entrée. Une fois le

vecteur latent de l'état obtenu, celui-ci est concaténé avec l'action. Le vecteur résultant est ensuite traité par un réseau entièrement connecté composé de deux couches cachées. Cette dernière couche produit une valeur scalaire qui correspond à l'estimation de $Q_\phi(\mathbf{s}, \mathbf{a})$.

FIGURE 3 – Architecture générale des critic networks



Lors du calcul des cibles d'apprentissage et de la mise à jour de l'actor, SAC utilise la plus petite des deux estimations, ce qui rend l'apprentissage plus stable et limite le biais de surestimation.

2.3.3 Normalisation des observations

Les différentes composantes de l'état présentent des échelles de valeurs hétérogènes. Afin de faciliter l'apprentissage, elles sont normalisées avant d'être fournies aux réseaux de neurones.

- La distance à la cible est normalisée par la diagonale de l'environnement afin d'obtenir une valeur comprise entre 0 et 1.
- Les vitesses linéaire et angulaire sont normalisées par leurs valeurs maximales.
- Les variables angulaires (orientation du robot et erreur d'orientation par rapport à la cible) sont représentées par leurs composantes trigonométriques ($\cos(\theta)$, $\sin(\theta)$) afin d'éviter la discontinuité entre $-\pi$ et π .
- Les cartes locales étant binaires, elles ne nécessitent pas de normalisation supplémentaire.

3 Protocole expérimental

Cette section décrit le protocole expérimental utilisé pour entraîner et évaluer la politique de navigation. L'apprentissage repose sur un curriculum probabiliste composé de tâches de difficulté croissante. La politique obtenue est ensuite évaluée à partir de plusieurs indicateurs afin d'analyser ses performances, sa robustesse et sa capacité de généralisation.

3.1 Curriculum d'entraînement

L'objectif du curriculum est d'amener progressivement l'agent à résoudre des tâches de navigation de difficulté croissante. Plutôt que de confronter directement la politique aux scénarios les plus complexes, l'apprentissage est structuré autour d'un ensemble de tâches ordonnées (Bengio et al., 2009; Narvekar et al., 2020). Une stratégie probabiliste, inspirée de Gupta et al. (2017), est utilisée afin de privilégier progressivement les tâches les plus difficiles tout en continuant à sélectionner les tâches précédemment rencontrées.

3.1.1 Stratégie de curriculum probabiliste

La stratégie repose sur une distribution de probabilité définie sur l'ensemble des tâches du curriculum. Cette distribution est adaptée au cours de l'apprentissage afin de favoriser la tâche correspondant au niveau de difficulté actuellement visé.

Le curriculum est défini comme un ensemble ordonné de K tâches de difficulté croissante :

$$\mathcal{T} = (T_1, T_2, \dots, T_K).$$

L'objectif est de privilégier progressivement les tâches les plus complexes, tout en conservant une probabilité de sélectionner chaque tâche. À chaque étape du curriculum, une tâche T_f est définie comme la tâche actuellement privilégiée. Un vecteur de paramètres de Dirichlet κ est alors construit de manière à attribuer un poids plus important à cette tâche :

$$\kappa_i = \begin{cases} K & \text{si } i = f, \\ 1 & \text{sinon,} \end{cases} \quad i \in \llbracket 1, K \rrbracket.$$

Au début de l'apprentissage, la tâche privilégiée est la première tâche du curriculum, soit $f = 1$.

Un vecteur de probabilités est ensuite échantillonné à partir d'une distribution de Dirichlet :

$$\mathbf{w} \sim \text{Dirichlet}(\kappa), \quad \text{avec } \mathbf{w} = (w_1, w_2, \dots, w_K) \quad \text{et} \quad \sum_{i=1}^K w_i = 1.$$

Chaque composante w_i représente alors la probabilité de sélectionner la tâche T_i . Une tâche est finalement échantillonnée selon cette distribution : $i \sim \text{Categorical}(\mathbf{w})$.

La politique SAC est entraînée sur la tâche sélectionnée pendant un nombre fixe d'étapes, appelé

chunk. Cette opération est répétée pendant plusieurs chunks avant d’évaluer la politique courante sur l’ensemble des tâches du curriculum. Cette évaluation fournit un taux de réussite S_i pour chacune des tâches T_i .

La progression dans le curriculum dépend alors du taux de réussite obtenu sur la tâche actuellement privilégiée. Si celui-ci dépasse un seuil fixé ξ_{curr} , la tâche suivante devient la nouvelle tâche privilégiée. Le paramètre dominant de la distribution de Dirichlet est donc déplacé progressivement d’une tâche vers la suivante.

Lorsque la dernière tâche devient la tâche privilégiée, le curriculum n’est considéré comme terminé que si l’ensemble des tâches atteint le seuil de réussite fixé. Dans le cas contraire, l’entraînement revient vers la première tâche dont les performances restent insuffisantes. Cette procédure permet de progresser vers des tâches plus complexes tout en contrôlant le maintien des performances sur les tâches précédemment apprises.

Parallèlement, le taux de réussite moyen sur l’ensemble des tâches est utilisé pour suivre la performance globale de la politique :

$$S_{\text{global}} = \frac{1}{K} \sum_{i=1}^K S_i.$$

La politique obtenant la meilleure valeur de S_{global} est sauvegardée. L’apprentissage reste cependant limité par un budget maximal d’étapes fixé à l’avance, si celui-ci est atteint, la dernière politique politique entraînée est également sauvegardée.

3.1.2 Scénarios d’entraînement

Le curriculum est constitué de seize tâches d’apprentissage, présentées dans la Table 1 et illustrées par la Figure 4. Les scénarios sont caractérisés par la largeur et la hauteur de l’environnement (W_{env} et H_{env}), le nombre d’agents (N_a), d’obstacles statiques (N_s), d’obstacles dynamiques (N_m), d’objectifs à atteindre (N_g) ainsi que par le nombre maximal d’étapes par épisode (T_{max}).

TABLE 1 – Récapitulatif des scénarios d’entraînement

<i>Tâche</i>	<i>Objectif</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
T_1	Apprendre à rejoindre une cible dans un environnement libre.	64	32	1	0	0	1	100
T_2	Apprendre un premier comportement de contournement avec quelques obstacles statiques fixes.	64	32	1	3	0	1	100
T_3	Renforcer l’évitement d’obstacles dans un environnement fixe plus encombré.	64	32	1	6	0	1	100
T_4	Renforcer l’évitement d’obstacles dans un environnement fixe encore plus encombré.	64	32	1	9	0	1	100

<i>Tâche</i>	<i>Objectif</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
T_5	Généraliser les comportements d'évitement à des configurations aléatoires et denses.	64	32	1	9	0	1	100
T_6	Introduire l'évitement d'obstacles dynamiques dans un environnement de type aéroport.	64	32	1	0	3	1	100
T_7	Combiner l'évitement d'obstacles statiques et dynamiques dans un environnement de type aéroport.	64	32	1	2	3	1	150
T_8	Apprendre à naviguer dans un environnement aléatoire dense comportant plusieurs objectifs et des obstacles statiques et dynamiques.	64	32	1	9	2	2	200
T_9	Introduire un environnement structuré de type entrepôt avec des obstacles dynamiques.	64	32	1	10	2	1	200
T_{10}	Augmenter la densité et la complexité de l'environnement de type entrepôt.	64	32	1	12	2	1	250
T_{11}	Augmenter encore une fois la densité et la complexité de l'environnement de type entrepôt.	64	32	1	14	2	1	250
T_{12}	Introduire un environnement hospitalier structuré avec plusieurs objectifs et un obstacle dynamique.	64	32	1	8	1	2	250
T_{13}	Introduire la navigation multi-agent dans un scénario de trajectoires croisées.	64	32	4	0	2	1	150
T_{14}	Apprendre la navigation multi-agent dans un environnement de type entrepôt dense.	64	32	2	14	2	1	150
T_{15}	Apprendre la navigation multi-agent dans un environnement de type aéroport.	64	32	5	0	0	1	200
T_{16}	Apprendre la navigation multi-agent dans un environnement de type aéroport comportant un obstacle statique et un obstacle dynamique.	64	32	5	1	1	2	200

Tâche 1 – Navigation de base : Dans un premier temps, l'agent évolue dans un environnement entièrement libre de tout obstacle. L'objectif est d'apprendre les bases de la navigation, c'est-à-dire rejoindre deux cibles en minimisant le temps de trajet tout en contrôlant correctement sa vitesse linéaire et sa vitesse angulaire. Cette première étape permet également de valider le bon

fonctionnement de la fonction de récompense pour la progression vers l'objectif avant d'introduire des contraintes supplémentaires.

Tâche 2 – Introduction des obstacles statiques : Trois obstacles statiques sont introduits dans une configuration fixe. Les positions de l'agent, de la cible et des obstacles sont déterminées de manière à imposer un premier contournement. L'objectif est d'apprendre à s'écarter de la trajectoire directe lorsque celle-ci est bloquée, tout en continuant à progresser vers la cible.

Tâche 3 – Navigation dans un environnement fixe plus encombré : La difficulté est augmentée en portant le nombre d'obstacles statiques à six. La configuration reste fixe afin de permettre à l'agent de consolider ses comportements d'évitement dans un environnement plus contraint.

Tâche 4 – Navigation dans un environnement fixe encore plus encombré : La difficulté est encore augmentée en portant le nombre d'obstacles statiques à neuf, réduisant les possibilités de déplacement et imposant une navigation plus précise.

Tâche 5 – Généralisation à des configurations statiques aléatoires : Le nombre d'obstacles statiques est conservé, mais leur configuration est générée aléatoirement à chaque épisode. Cette tâche vise à éviter la mémorisation des configurations fixes précédentes et à développer une stratégie de navigation capable de s'adapter à des environnements inconnus.

Tâche 6 – Introduction des obstacles dynamiques : L'agent est placé dans un environnement de type aéroport comportant trois obstacles dynamiques. Il doit désormais tenir compte de l'évolution future de la scène et adapter sa trajectoire aux déplacements des obstacles. Cette tâche introduit les premiers comportements d'anticipation et d'évitement dynamique.

Tâche 7 – Combinaison d'obstacles statiques et dynamiques : Deux obstacles statiques sont ajoutés à l'environnement de type aéroport, tandis que les trois obstacles dynamiques sont conservés. L'agent doit ainsi gérer simultanément des contraintes fixes et des interactions mobiles.

Tâche 8 – Généralisation à des configurations dynamiques aléatoires : L'agent évolue dans un environnement aléatoire comportant des obstacles statiques et dynamiques. Cette tâche combine une forte variabilité spatiale et des interactions dynamiques. L'agent doit conserver un comportement efficace au cours de plusieurs phases de navigation au sein d'un même épisode.

Tâche 9 – Navigation dans un environnement de type entrepôt : L'agent est ensuite confronté à un environnement structuré de type entrepôt, composé de plusieurs obstacles statiques et dynamiques. La présence d'allées et de passages étroits limite les trajectoires possibles. Cette tâche vise à apprendre à naviguer dans une structure organisée tout en tenant compte des déplacements d'autres entités. La densité de cet entrepôt est augmentée durant la **Tâche 10**.

Tâche 12 – Navigation dans un environnement de type hospitalier : L'agent évolue dans un environnement représentant un couloir principal desservant plusieurs salles, dans lequel circule également un obstacle dynamique. Cette structure impose une navigation dans des espaces contraints, caractérisés notamment par des couloirs étroits et des zones d'intersection. Cette tâche introduit ainsi un nouveau contexte d'application. *NB : dans cette configuration, le nombre d'obstacles statiques correspond au nombre de salles.*

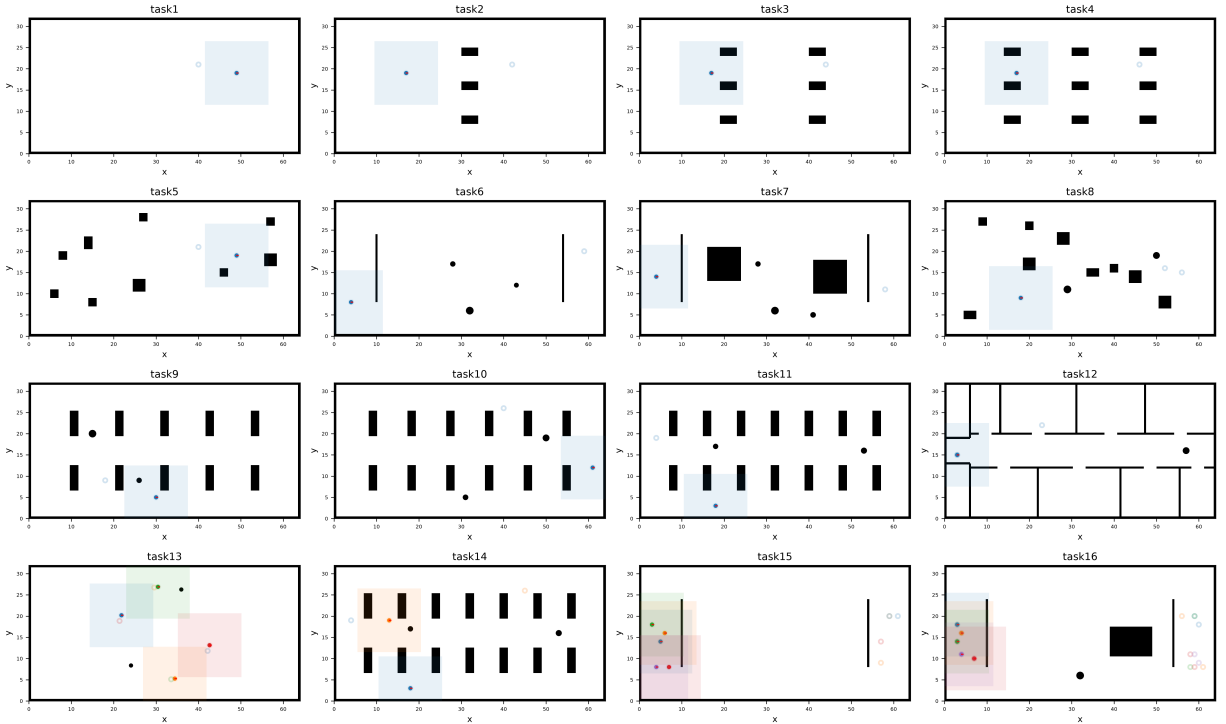
Tâche 13 – Introduction des interactions multi-agents : La navigation multi-agent est introduite dans un scénario de type crossing comprenant quatre agents et deux obstacles dynamiques. Les trajectoires sont amenées à se croiser, ce qui augmente le risque de collision et nécessite d’adapter la vitesse ou la direction lorsque plusieurs déplacements entrent en conflit. L’objectif est ainsi d’apprendre à l’agent entraîné à tenir compte des autres agents lors de la sélection de ses actions.

Tâche 14 – Navigation multi-agent dans un environnement de type entrepôt : Deux agents évoluent simultanément dans un environnement de type entrepôt comportant deux obstacles dynamiques. Cette tâche combine la forte densité de l’environnement structuré avec les interactions entre agents. Les passages étroits augmentent les risques de blocage et nécessitent des comportements d’évitement plus coordonnés.

Tâche 15 – Navigation multi-agent dans un environnement aéroportuaire fixe : Le nombre d’agents est porté à cinq dans un environnement de type aéroport. Cette configuration permet de concentrer l’apprentissage sur les interactions entre agents. L’objectif est d’apprendre à maintenir une navigation efficace lorsque plusieurs agents partagent le même espace.

Tâche 16 – Navigation multi-agent dans un environnement aéroportuaire dynamique : Un obstacle statique et un obstacle dynamique sont intégrés à la précédente configuration. Cette dernière tâche combine les principales difficultés introduites au cours du curriculum : navigation multi-agent, obstacles fixes, interactions dynamiques et objectifs successifs.

FIGURE 4 – Scénarios types utilisés pour le curriculum d’entraînement.



Dans l’ensemble des configurations, les obstacles dynamiques suivent des trajectoires prédéfinies, calculées à l’aide de l’algorithme A*. Ils ne sont donc pas contrôlés par la politique SAC. Dans les scénarios multi-agents, un seul agent contribue directement à l’apprentissage de la politique, tandis que les autres agents appliquent de manière déterministe la politique courante. Cette organisation

permet d'introduire des interactions entre agents sans entraîner séparément une politique pour chacun d'eux. Durant l'évaluation, l'ensemble des agents applique de manière déterministe la politique finale obtenue.

Les principales configurations d'environnement mobilisées dans le curriculum (random, crossing, warehouse, hospital et airport) sont présentées plus en détail dans la section suivante.

3.2 Environnements de test

Une fois l'entraînement terminé, la politique retenue est évaluée sur un ensemble de scénarios distincts de ceux utilisés pour le curriculum. Ces environnements doivent permettre d'étudier à la fois les performances de navigation dans des situations proches de l'entraînement et la capacité de la politique à généraliser à des configurations différentes ou plus complexes.

La politique est évaluée sur dix tâches, présentées dans la Table 2 et illustrées par la Figure 5.

TABLE 2 – Récapitulatif des scénarios d'évaluation

<i>Éval.</i>	<i>Description</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
E_1	Navigation mono-agent dans un environnement aléatoire comportant des obstacles statiques et dynamiques.	64	32	1	6	2	2	200
E_2	Navigation de deux agents dans un environnement aléatoire comportant des obstacles statiques et dynamiques.	64	32	2	6	2	2	200
E_3	Navigation de deux agents dans un environnement de type entrepôt comportant des obstacles dynamiques.	64	32	2	14	2	1	250
E_4	Navigation multi-agents dans un environnement caractérisé par des trajectoires fortement conflictuelles.	64	32	5	0	1	1	200
E_5	Navigation mono-agent dans un environnement hospitalier comportant plusieurs salles et un obstacle dynamique.	64	32	1	8	1	3	350
E_6	Navigation mono-agent dans un environnement hospitalier comportant plusieurs salles et un obstacle dynamique, en augmentant la densité afin d'avoir des chemins plus étroits.	64	32	1	12	1	3	350
E_7	Navigation de deux agents dans un environnement hospitalier comportant plusieurs salles et un obstacle dynamique.	64	32	2	12	2	3	350

<i>Éval.</i>	<i>Description</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
E_8	Navigation de huit agents dans un environnement aéroportuaire comportant un obstacle dynamique.	64	32	8	0	1	1	200
E_9	Navigation de vingt agents dans un environnement aéroportuaire de plus grande taille.	128	64	20	0	0	1	300
E_{10}	Navigation de 50 agents dans un environnement aéroportuaire de plus grande taille.	256	128	50	0	0	1	400

Scénarios aléatoires — random : Les scénarios aléatoires sont générés automatiquement à chaque épisode en faisant varier la position des obstacles, de l’agent et de la cible, tout en respectant les contraintes de génération (absence de recouvrement, distance minimale entre les objets, etc.). Ils permettent d’évaluer la capacité de l’agent à généraliser à des configurations qu’il n’a jamais rencontrées au cours de l’entraînement. La difficulté peut être ajustée en faisant varier le nombre d’obstacles statiques et dynamiques. La taille des obstacles statiques est déterminée de manière aléatoire selon la taille de l’environnement et le nombre d’obstacles demandé. La taille des obstacles dynamiques est également générée aléatoirement selon des bornes minimale et maximale données.

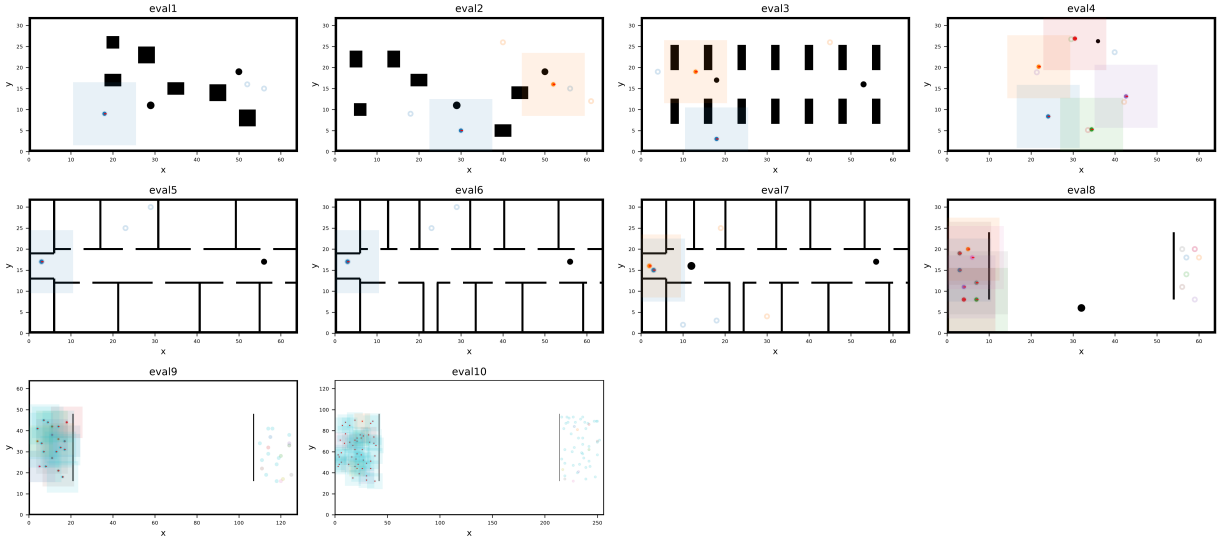
Scénarios de type entrepôt -- warehouse : Les scénarios de type warehouse reproduisent un environnement structuré composé d’allées séparées par des obstacles rectangulaires représentant des rayonnages. Ils permettent d’évaluer la capacité de l’agent à planifier des trajectoires dans des espaces contraints, où les possibilités de déplacement sont limitées et où les passages sont plus étroits. Ce type d’environnement se rapproche des configurations rencontrées dans les systèmes automatisés de transport. La taille des obstacles statiques est déterminée de manière aléatoire selon la taille de l’environnement et le nombre d’obstacles demandé. La taille des obstacles dynamiques est également générée aléatoirement selon des bornes minimale et maximale données.

Scénarios de type circulaire – crossing : Les scénarios de type crossing sont conçus pour produire des trajectoires conflictuelles. Les agents et les obstacles dynamiques sont disposés de manière à traverser l’environnement selon des directions opposées ou convergentes, ce qui entraîne de nombreux croisements de trajectoires. Ces configurations permettent d’étudier les comportements d’évitement dans des situations où le risque de collision est élevé et où les décisions prises par une entité influencent directement les possibilités de déplacement des autres. La taille des obstacles dynamiques correspond ici à celle des agents.

Scénarios de type hospitalier – hospital : Les scénarios de type hospital représentent un couloir principal desservant plusieurs salles, modélisées par des obstacles statiques. Des obstacles dynamiques peuvent également circuler dans le couloir. Cette configuration impose une navigation dans des espaces relativement étroits, où les possibilités de dépassement et de contournement sont limitées. Elle permet d’évaluer la capacité de la politique à se déplacer de manière sûre et fluide dans un environnement structuré. L’agent apparaît à une position prédéfinie correspondant à un ascenseur, se déplace vers une cible située dans l’une des salles, puis revient à sa position initiale.

Scénarios de type aéroportuaire – airport : Les scénarios de type airport représentent un environnement ouvert dans lequel plusieurs agents doivent se déplacer entre des zones de départ et d’arrivée situées dans des parties distinctes de la carte. Ces scénarios sont principalement utilisés pour étudier les interactions multi-agents et le passage à l’échelle, en augmentant progressivement le nombre d’agents ainsi que les dimensions de l’environnement. Ils se rapprochent du contexte d’application final, dans lequel plusieurs véhicules autonomes doivent partager un même espace de circulation afin de prendre en charge et de déposer des bagages. Le nombre d’objectifs correspond ici au nombre de bagages à déposer. Après avoir atteint chaque objectif, l’agent retourne à sa position initiale dans la zone de ramassage.

FIGURE 5 – Scénarios types utilisés pour l’évaluation.



3.3 Critères d’évaluation

Afin d’évaluer les performances de la politique apprise, plusieurs indicateurs sont considérés. **L’objectif est d’analyser à la fois la capacité de l’agent à atteindre sa cible, mais également l’efficacité des trajectoires produites et le respect des contraintes de sécurité.**

- Le principal indicateur est le taux de réussite (success rate), défini comme la proportion d’épisodes pour lesquels l’agent atteint l’ensemble de ses objectifs avant la fin de l’épisode. Cette métrique permet d’évaluer directement la capacité de l’algorithme à accomplir la tâche de navigation.
- Le taux de collision (collision rate) correspond à la proportion d’épisodes interrompus à la suite d’une collision avec un obstacle. Il permet de mesurer le respect des contraintes de sécurité imposées au robot.
- Le temps moyen de trajet (mean travel time) est également considéré. Il correspond au nombre moyen d’étapes nécessaires pour atteindre la cible. Cette métrique permet d’évaluer l’efficacité de la navigation en complément du taux de réussite, deux politiques pouvant atteindre la cible avec des temps de parcours très différents.
- Enfin, le retour cumulé moyen (average return) est suivi au cours de l’apprentissage. Il correspond à la somme des récompenses obtenues au cours d’un épisode, moyennée sur plusieurs

épisodes. Cet indicateur permet d'observer la convergence de l'algorithme et d'analyser l'influence de la fonction de récompense sur les comportements appris

En plus de ces indicateurs globaux, une analyse plus fine sera réalisée en étudiant séparément les différentes composantes de la fonction de récompense (progression, sécurité, rotation et collision). Cette décomposition permettra de mieux comprendre les comportements développés par l'agent au cours de l'apprentissage et d'identifier l'influence de chaque terme de récompense sur la politique obtenue.

4 Résultats

Cette section présente les premiers résultats obtenus avec le modèle proposé. L’objectif est d’évaluer la capacité de l’algorithme à apprendre une politique de navigation efficace à partir du curriculum d’entraînement défini précédemment, puis d’analyser les comportements développés par l’agent dans les différents scénarios. Les performances sont étudiées à l’aide des critères d’évaluation présentés précédemment.

L’entraînement des modèles a été réalisé sur une machine équipée d’un processeur Intel Core i7-13700H, de 16Go de mémoire vive et d’une carte graphique NVIDIA GeForce RTX 4070 Laptop GPU (8 Go). À titre indicatif, le temps d’entraînement est d’environ ... pour ... épisodes (... itérations), dans les conditions expérimentales retenues.

4.1 Validation de l’apprentissage

4.2 Analyse des scénarios d’entraînement

Pour chaque expérience, les principaux indicateurs de performance sont relevés sur la politique finale obtenue après l’entraînement. Ces résultats permettent d’évaluer la capacité de l’agent à résoudre le scénario considéré avant d’introduire un niveau de difficulté supplémentaire.

TABLE 3 – Performances obtenues à l’issue de l’entraînement

<i>Tâche</i>	<i>N_{ep}</i>	<i>Succès</i>	<i>Collision</i>	<i>Timeout</i>	<i>Vitesse moyenne</i>	<i>Retour moyen</i>
<i>T₁</i>	—	—	—	—	—	—
<i>T₂</i>	—	—	—	—	—	—
<i>T₃</i>	—	—	—	—	—	—
<i>T₄</i>	—	—	—	—	—	—
<i>T₅</i>	—	—	—	—	—	—
<i>T₆</i>	—	—	—	—	—	—
<i>T₇</i>	—	—	—	—	—	—
<i>T₈</i>	—	—	—	—	—	—
<i>T₉</i>	—	—	—	—	—	—
<i>T₁₀</i>	—	—	—	—	—	—
<i>T₁₁</i>	—	—	—	—	—	—
<i>T₁₂</i>	—	—	—	—	—	—
<i>T₁₃</i>	—	—	—	—	—	—
<i>T₁₄</i>	—	—	—	—	—	—
<i>T₁₅</i>	—	—	—	—	—	—
<i>T₁₆</i>	—	—	—	—	—	—

Lecture du tableau :

4.3 Évaluation de la politique

TABLE 4 – Performances obtenues à l’issue de l’évaluation

<i>Éval.</i>	<i>N_{ép}</i>	<i>Succès</i>	<i>Collision</i>	<i>Timeout</i>	<i>Vitesse moyenne</i>	<i>Retour moyen</i>
<i>E₁</i>	—	—	—	—	—	—
<i>E₂</i>	—	—	—	—	—	—
<i>E₃</i>	—	—	—	—	—	—
<i>E₄</i>	—	—	—	—	—	—
<i>E₅</i>	—	—	—	—	—	—
<i>E₆</i>	—	—	—	—	—	—
<i>E₇</i>	—	—	—	—	—	—
<i>E₈</i>	—	—	—	—	—	—
<i>E₉</i>	—	—	—	—	—	—
<i>E₁₀</i>	—	—	—	—	—	—

Lecture du tableau :

Glossaire

TABLE 5 – Récapitulatif des principaux paramètres du protocole expérimental

<i>Paramètre</i>	<i>Description</i>	<i>Valeur</i>
<i>Environnement</i>		
W_{env}	Largeur de l’environnement	64 m
H_{env}	Hauteur de l’environnement	32 m
Δt	Durée d’une étape de simulation	1 s
T_{max}	Nombre maximal d’étapes par épisode	—
N_a	Nombre d’agents	—
N_s	Nombre d’obstacles statiques	—
N_m	Nombre d’obstacles dynamiques	—
N_g	Nombre d’objectifs	—
r_{min}	Rayon minimal des obstacles	0.5 m
r_{max}	Rayon maximal des obstacles	1.0 m
<i>Agent</i>		
K	Nombre de cartes locales empilées	3
W	Largeur du champ de vision local	9 m
H	Hauteur du champ de vision local	9 m
r	Rayon du robot	0.5 m
v_{min}	Vitesse linéaire minimale	0 m/s
v_{max}	Vitesse linéaire maximale	5 m/s
ω_{min}	Vitesse angulaire minimale	$-\pi/3$ rad/s
ω_{max}	Vitesse angulaire maximale	$\pi/3$ rad/s
ξ_c	Seuil de collision	0.5 m
ξ_g	Seuil d’atteinte de la cible	0.5 m
<i>Fonction de récompense</i>		
β_1	Poids de la récompense de progression	5.0
β_2	Poids de la pénalité de rotation	0.1
β_3	Poids de la pénalité de sécurité	0.5
$r_{\text{goal_reward}}$	Bonus attribué lorsque la cible est atteinte	400
$r_{\text{collision_malus}}$	Pénalité appliquée en cas de collision	-100
$r_{\text{angular_malus}}$	Coefficient de pénalité angulaire	-2
$r_{\text{safety_malus}}$	Coefficient de pénalité de sécurité	-1
$r_{\text{linear_bonus}}$	Bonus proportionnel à la vitesse linéaire	2
$r_{\text{step_malus}}$	Pénalité appliquée à chaque étape	-1
$r_{\text{progress_reduction}}$	Facteur de réduction de la récompense de progression à proximité d’un obstacle	0.2
ξ_s	Distance minimale de sécurité	2.0 m

<i>Paramètre</i>	<i>Description</i>	<i>Valeur</i>
<i>Algorithme SAC</i>		
γ	Facteur d'actualisation	0.990
α	Coefficient d'entropie	0.150
λ	Coefficient de soft update	0.005
η_{actor}	Coefficient d'apprentissage de l'actor network	3×10^{-5}
η_{critic}	Coefficient d'apprentissage des critic networks	3×10^{-4}
N_{batch}	Taille du mini-batch	64
N_{memory}	Taille maximale du replay buffer	100 000

Références

- Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum Learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.
- G. Chen, S. Yao, J. Ma, L. Pan, Y. Chen, P. Xu, J. Ji, and X. Chen. Distributed Non-Communicating Multi-Robot Collision Avoidance via Map-Based Deep Reinforcement Learning. *Sensors*, 20(17):4836, 2020. doi : 10.3390/s20174836.
- J. Choi, G. Lee, and C. Lee. Reinforcement learning-based dynamic obstacle avoidance and integration of path planning. *International Journal of Intelligent Robotics and Applications*, 14(5):663–677, 2021.
- J. K. Gupta, M. Egorov, and M. J. Kochenderfer. Cooperative Multi-agent Control Using Deep Reinforcement Learning. In *AAMAS Workshops*, 2017. URL <https://api.semanticscholar.org/CorpusID:9421360>.
- T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft Actor-Critic : Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Z. Huang, Z. Ren, T. Chen, S. Cai, and C. Xu. Autonomous navigation and collision avoidance for agv in dynamic environments : An enhanced deep reinforcement learning approach with composite rewards and dynamic update mechanisms. *IET Cyber-Systems and Robotics*, 2024.
- D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes, 2022. URL <https://arxiv.org/abs/1312.6114>.
- P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan. Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone. Curriculum Learning for Reinforcement Learning domains : A framework and survey, 2020. URL <https://arxiv.org/abs/2003.04960>.
- OpenAI. Soft actor-critic, 2023. URL <https://spinningup.openai.com/en/latest/algorithms/sac.html>. Spinning Up documentation, consulté le 5 mai 2026.
- H. van Hasselt. Double Q-learning. In J. D. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. S. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23, pages 2613–2621. Curran Associates, Inc., 2010.

Table des figures

1	Architecture générale de l'extracteur de caractéristiques convolutionnel	9
2	Architecture générale de l'actor network	10
3	Architecture générale des critic networks	11
4	Scénarios types utilisés pour le curriculum d'entraînement.	16
5	Scénarios types utilisés pour l'évaluation.	19
6	Évolution des performances au cours de l'entraînement (Expérience 3)	28
7	Évolution des performances au cours de l'entraînement (Expérience 7)	28

Liste des tableaux

1	Récapitulatif des scénarios d'entraînement	13
2	Récapitulatif des scénarios d'évaluation	17
3	Performances obtenues à l'issue de l'entraînement	21
4	Performances obtenues à l'issue de l'évaluation	22
5	Récapitulatif des principaux paramètres du protocole expérimental	23

A Annexes

A.1 Pseudo-code de l'algorithme SAC

Algorithm 1: Soft Actor-Critic

inputs :

θ : paramètres initiaux de la politique
 ϕ_1, ϕ_2 : paramètres initiaux des fonctions de valeurs d'action
 $\mathcal{D}$: replay buffer vide
 $\eta_{\text{critic}}, \eta_{\text{actor}}$: taux d'apprentissage

initialisation : $\phi_{1,\text{target}} \leftarrow \phi_1, \phi_{2,\text{target}} \leftarrow \phi_2$

On applique la procédure jusqu'à convergence...

for chaque itération **do**

for chaque étape dans l'environnement **do**

On observe s puis on exécute a dans l'environnement

$\mathbf{a} \sim \pi_{\theta}(\cdot | \mathbf{s})$

$\mathbf{s}' \sim P(\cdot | \mathbf{s}, \mathbf{a})$

$r \leftarrow R(\mathbf{s}, \mathbf{a})$

On stocke les transitions dans le replay buffer

$\mathcal{D} \leftarrow \mathcal{D} \cup (\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$

Si s' est terminal, on recommence

if $d = 1$ **then** réinitialiser l'environnement

for chaque étape de mise à jour **do**

Échantillonner un batch de transitions aléatoirement

$B \sim \mathcal{D}$

Calculer les targets des fonctions Q

$\tilde{\mathbf{a}}' \sim \pi_{\theta}(\cdot | \mathbf{s}')$

$y \leftarrow r + \gamma(1 - d) \left(\min_{i \in \{1, 2\}} Q_{\phi_{i,\text{target}}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}') \right)$

Mettre à jour Q_{ϕ_1} et Q_{ϕ_2} par descente de gradient, avec :

$\phi_i \leftarrow \phi_i - \eta_{\text{critic}} \nabla_{\phi_i} \frac{1}{|B|} \sum_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \in B} (Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y)^2$

Mettre à jour π_{θ} par montée de gradient, avec :

$\varepsilon \sim \mathcal{N}(0, I)$

$\tilde{\mathbf{a}}_{\theta}(\mathbf{s}, \varepsilon) \leftarrow \tanh(\mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s}) \odot \varepsilon)$

$\theta \leftarrow \theta + \eta_{\text{actor}} \nabla_{\theta} \frac{1}{|B|} \sum_{\mathbf{s} \in B} (\min_{i \in \{1, 2\}} Q_{\phi_i}(\mathbf{s}, \tilde{\mathbf{a}}_{\theta}) - \alpha \log \pi_{\theta}(\tilde{\mathbf{a}}_{\theta} | \mathbf{s}))$

Mettre à jour les target networks :

$\phi_{1,\text{target}} \leftarrow (1 - \lambda)\phi_{1,\text{target}} + \lambda\phi_1$

$\phi_{2,\text{target}} \leftarrow (1 - \lambda)\phi_{2,\text{target}} + \lambda\phi_2$

A.2 Compléments sur l'entraînement

FIGURE 6 – Évolution des performances au cours de l'entraînement (Expérience 3)

FIGURE 7 – Évolution des performances au cours de l'entraînement (Expérience 7)